

Proyecto Integrador

Especificaciones del Reto: Diseño y Entrenamiento de un Mini-LLM

Diplomado en Inteligencia Artificial

1. Objetivo del reto

El objetivo de este proyecto integrador es que el participante diseñe, implemente y evalúe un **modelo de lenguaje a pequeña escala (mini-LLM)**, integrando de forma práctica los conceptos vistos en el módulo de *Entrenamiento de Modelos Grandes de Lenguaje*.

Al finalizar el reto, el estudiante deberá demostrar dominio de:

- preparación y tokenización de datos textuales,
- formulación del problema de *next-token prediction*,
- entrenamiento auto-regresivo de modelos de lenguaje,
- evaluación cuantitativa mediante métricas estándar,
- generación de texto y análisis de resultados,
- toma de decisiones de ingeniería en el entrenamiento de modelos.

2. Alcance general

El proyecto consiste en construir un modelo de lenguaje que, dado un fragmento de texto de entrada (*prompt*), sea capaz de generar texto de manera auto-regresiva, aprendiendo patrones estadísticos del lenguaje a partir de un conjunto de datos textual.

El modelo desarrollado **no busca competir con LLMs comerciales**, sino servir como una plataforma experimental para comprender sus principios fundamentales.

3. Requisitos mínimos obligatorios

Todos los equipos o participantes deberán cumplir con los siguientes requisitos mínimos.

3.1. Datos

- Utilizar un archivo de texto plano (`.txt`) como conjunto de datos.
- Tamaño recomendado del dataset: entre **10 KB y 500 KB**.
- Documentar claramente el origen del texto (libro, artículo, documentación técnica, texto propio, etc.).
- Aplicar, en caso necesario, una limpieza mínima de caracteres (normalización de saltos de línea, eliminación de símbolos no deseados).

3.2. Tokenización

- La tokenización deberá ser **carácter a carácter**.
- Se deberá construir explícitamente:
 - un vocabulario de símbolos,
 - un mapeo símbolo $\rightarrow$ identificador,
 - un mapeo identificador $\rightarrow$ símbolo.

3.3. Modelo

El participante deberá implementar **al menos una** de las siguientes opciones:

- **Modelo baseline**: red recurrente (RNN o GRU).
- **Modelo avanzado**: mini-Transformer causal con self-attention enmascarada.

El modelo deberá:

- operar bajo el esquema de *next-token prediction*,
- respetar la causalidad (no acceso a tokens futuros),
- producir una distribución de probabilidad sobre el vocabulario.

3.4. Entrenamiento

- Función de pérdida: **entropía cruzada**.
- Optimizador: **Adam** o **AdamW**.
- Entrenamiento por mini-batches.
- Registro explícito de:
 - pérdida de entrenamiento (`train_loss`),
 - pérdida de validación (`val_loss`),
 - número de épocas.

3.5. Evaluación

- Reportar la pérdida de validación final.
- Calcular la perplejidad aproximada:

$$\text{Perplejidad} = \exp(\text{val_loss})$$

cuando la pérdida lo permita.

3.6. Generación de texto

- Definir un **prompt fijo**.
- Generar texto con al menos **tres valores distintos de temperatura** (por ejemplo: baja, media y alta).
- Longitud sugerida de la generación: entre 300 y 600 tokens.

3.7. Reproducibilidad

- Fijar semillas aleatorias.
- Guardar la configuración completa del experimento (hiperparámetros, arquitectura y rutas).

4. Extensiones opcionales

Las siguientes actividades son opcionales, pero fortalecen el proyecto:

- Comparar dos arquitecturas (por ejemplo, GRU vs Transformer).
- Analizar el efecto del tamaño del contexto (`seq_len`).
- Probar distintos tamaños de modelo (small vs medium).
- Aplicar técnicas de entrenamiento eficiente:
 - acumulación de gradientes,
 - precisión mixta (AMP),
 - gradient checkpointing.
- Realizar un pequeño estudio de hiperparámetros.

5. Resultados esperados

Al finalizar el reto, se espera que el participante sea capaz de:

- explicar cómo se entrena un modelo de lenguaje auto-regresivo,
- interpretar métricas de entrenamiento y validación,
- justificar decisiones de arquitectura y entrenamiento,
- identificar limitaciones prácticas de los LLMs,
- proponer mejoras fundamentadas en datos y cómputo.